

To install typeorm for nestjs: `npm install --save @nestjs/typeorm typeorm mysql2`

Then to use it with app module:

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mariadb',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: '123',
      database: 'test',
      entities: [],
      synchronize: true
    })
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
*****
```

WARNING

Setting `synchronize: true` shouldn't be used in production - otherwise you can lose production data.

The `forRoot()` method supports all the configuration properties exposed by the `DataSource` constructor from the **TypeORM** package. In addition, there are several extra configuration properties described below.

<code>retryAttempts</code>	Number of attempts to connect to the database (default: <code>10</code>)
<code>retryDelay</code>	Delay between connection retry attempts (ms) (default: <code>3000</code>)
<code>autoLoadEntities</code>	If <code>true</code> , entities will be loaded automatically (default: <code>false</code>)

Once this is done, the TypeORM `DataSource` and `EntityManager` objects will be available to inject across the entire project (without needing to import any modules), for example:

app.module.ts

JS

```
import { DataSource } from 'typeorm';

@Module({
  imports: [TypeOrmModule.forRoot(), UsersModule],
})
export class AppModule {
  constructor(private dataSource: DataSource) {}
}
```

To define the User entity:

```
import { Column, Entity, PrimaryGeneratedColumn } from "typeorm";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;
  @Column()
  firstName: string;
  @Column()
  lastName: string;
  @Column({ default: true })
  isActive: boolean;
}
```

Then we import it in entities:

```
import { Module } from '@nestjs/common';
import { AppController } from '../app.controller';
import { AppService } from '../app.service';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../users/user.entity';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mariadb',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: '123',
      database: 'test',
      entities: [User],
      synchronize: true
    })
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}

*****
```

The to define the Users module:

```
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from './user.entity';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService]
})
export class UsersModule {}

*****
```

This module uses the `forFeature()` method to define which repositories are registered in the current scope.

With that in place, we can inject the `UsersRepository` into the `UsersService` using

the `@InjectRepository()` decorator:

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { User } from '../user.entity';
import { DeleteResult, Repository } from 'typeorm';
@Injectable()
export class UsersService {

  constructor(
    @InjectRepository(User) private userRepository: Repository<User>
  ) {}

  async findAll(): Promise<User[]> {
    return this.userRepository.find()
  }

  async findOne(id: number) :Promise<User | null> {
    return this.userRepository.findOneBy({ id })
  }

  async remove(id: number): Promise<DeleteResult> {
    return this.userRepository.delete(id)
  }
}
```

Relations

Relations are associations established between two or more tables. Relations are based on common fields from each table, often involving primary and foreign keys.

There are three types of relations:

One-to-one	Every row in the primary table has one and only one associated row in the foreign table. Use the <code>@OneToOne()</code> decorator to define this type of relation.
One-to-many / Many-to-one	Every row in the primary table has one or more related rows in the foreign table. Use the <code>@OneToMany()</code> and <code>@ManyToOne()</code> decorators to define this type of relation.
Many-to-many	Every row in the primary table has many related rows in the foreign table, and every record in the foreign table has many related rows in the primary table. Use the <code>@ManyToMany()</code> decorator to define this type of relation.

To autoload entities:

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'mariadb',
      host: 'localhost',
      port: 3306,
      username: 'root',
      password: '123',
      database: 'test',
      autoLoadEntities: true,
      entities: [Photo],
      synchronize: true
    }),
    UsersModule
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
*****
```

And to define one to many && many to one relation:

```
import { User } from "src/users/user.entity";
import { Column,ManyToOne, OneToOne, PrimaryGeneratedColumn } from "typeorm";

export class Photo {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  path: string;

  @ManyToOne(() => User, (user) => user.photos)
  user: User;
}
```

```

import { Photo } from "src/photo/photo.entity";
import { Column, Entity, OneToMany, PrimaryGeneratedColumn } from "typeorm";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  firstName: string;

  @Column()
  lastName: string;

  @Column({ default: true })
  isActive: boolean;

  @OneToMany(() => Photo, photo => photo.user)
  photos: Photo[]
}

*****

```

WARNING

Note that entities that aren't registered through the `forFeature()` method, but are only referenced from the entity (via a relationship), won't be included by way of the `autoLoadEntities` setting.

With that option specified, every entity registered through the `forFeature()` method will be automatically added to the `entities` array of the configuration object.

WARNING

Note that entities that aren't registered through the `forFeature()` method, but are only referenced from the entity (via a relationship), won't be included by way of the `autoLoadEntities` setting.

To separate the entity definitions instead of use annotations with entities classes:

For user.schema.ts:

```
import { EntitySchema } from 'typeorm';
import { User } from '../user.entity';

export const UserSchema = new EntitySchema<User>({
  name: 'User',
  target: User,
  columns: {
    id: {
      type: Number,
      primary: true,
      generated: true,
    },
    firstName: {
      type: String,
      nullable: false,
    },
    lastName: {
      type: String,
      nullable: false,
    },
    isActive: {
      type: Boolean,
      default: true,
    },
  },
  relations: {
    photos: {
      type: 'one-to-many',
      target: 'Photo'
    }
  }
})

*****
```

WARNING

If you provide the `target` option, the `name` option value has to be the same as the name of the target class. If you do not provide the `target` you can use any name.

```
@Module({
  imports: [TypeOrmModule.forFeature([UserSchema])],
  controllers: [UsersController],
  providers: [UsersService]
})
export class UsersModule {}
```
